pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Case Study: Organizing a Small Portfolio

WEEK 1 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

An email, six bonds, and no file

The handover

A credit desk takes over six corporate bonds from another team.

An email, with the holdings typed into the body

No file

No system record

Your job: store the holdings so it is easy to

Look up

everything about one bond

Update

a bond when its rating or price changes

Answer

simple questions about the portfolio

Hand on

the data in a structure built for analysis

The six bonds in the email

Ticker	Issuer	Sector	Rating	Score	Coupon (%)	Price	Face held
QVNE	Quorvane Energy	Energy	BBB	9	5.25	98.50	1,500,000
DNMR	Dunmarrow Rail	Transportation	BB+	11	6.125	103.25	500,000
PLWK	Pellwick Paper	Materials	A-	7	4.75	95.00	750,000
HLVF	Halvering Foods	Consumer	B+	14	7.50	101.00	250,000
OSTV	Ostrevale Telecom	Telecom	BBB-	10	5.75	99.00	1,000,000
VYRH	Veyrholt Health	Health Care	BB	12	6.75	102.50	400,000

AAA is 1 → Each notch lower adds one → **10 or lower: investment grade**

11 or higher: high yield

Start simple, improve when it gets awkward

1

Variables

One value each. It does not scale.

→

2

Lists

One column each. Look up and update by position.

→

3

Conditions and loops

Change the right bonds. Count what you need.

→

4

A function

A lookup in one line, written once.

→

5

A dictionary

Six lists gathered into one object.

→

6

A DataFrame

A table built for analysis.

Six Predict questions

Each shows code and asks what it will do.

Pause the video → **Decide** → **Then run the cell**

Find the position, then read across

In Python

```
pos = ticker_list.index('DNMR')
1
```

```
print(coupon_list[pos])
6.125
```

```
pos = ticker_list.index('OSTV')
score_list[pos] = 11
```

same job

same job

In Excel

```
=MATCH("DNMR", A2:A7, 0)
```

Finds the position: 2. MATCH counts from 1.

```
=INDEX(F2:F7, 2)
```

Reads the coupon column at that position: 6.125

Updating is the same move

OSTV is downgraded one notch, from BBB- to BB+. Its score moves from 10 to 11.

A correction for some bonds, not all

Every high yield price supplied was 0.5 too high

Ticker	Score	Before	After	Result
QVNE	9	98.5	98.5	Unchanged
DNMR	11	103.25	102.75	Corrected
PLWK	7	95.0	95.0	Unchanged
HLVF	14	101.0	100.5	Corrected
OSTV	11	99.0	98.5	Corrected
VYRH	12	102.5	102.0	Corrected

OSTV arrived with a score of 10. Its downgrade moved it to 11, so it is in the correction.

A condition picks the bonds. A loop reaches every one.

```
for i, score in enumerate(score_list):
    if score >= 11:
        price_list[i] = price_list[i] - 0.5
```

4 prices corrected

2 prices unchanged

The same loop would handle 200 bonds without a single change.

Counting is COUNTIF, written out

In Excel: one function

```
=COUNTIF(E2:E7, "<=10")
```

Ticker	Score	10 or lower?
QVNE	9	Yes
DNMR	11	No
PLWK	7	Yes
HLVF	14	No
OSTV	11	No
VYRH	12	No

In Python: the same count, written out

```
count = 0
for score in score_list:
    if score <= 10:
        count = count + 1
print('Number of investment grade bonds:', count)

Number of investment grade bonds: 2
```

Start a counter at zero → Add one each time the condition is true → Read the counter after the loop

Unlike COUNTIF, the loop can do anything else to each matching row.

Print shows it. Return hands it back.

Function 1 prints

```
def coupon_printed(ticker):
    pos = ticker_list.index(ticker)
    print(coupon_list[pos])

result = coupon_printed('QVNE')
print(result)

5.25
None
```

Function 2 returns

```
def coupon_returned(ticker):
    pos = ticker_list.index(ticker)
    return coupon_list[pos]

result = coupon_returned('QVNE')
print(result)

5.25
```

`result is None`

Shown on the screen, then gone. Nothing was handed back.

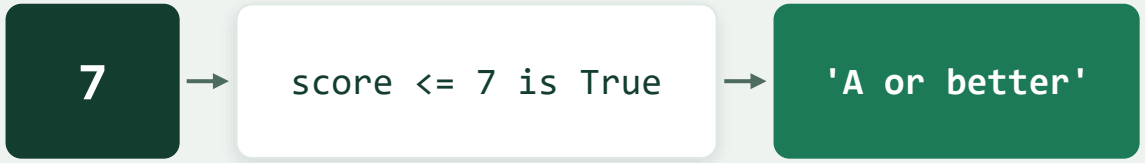
`result is 5.25`

Handed back, so it can be used in a calculation.

Python stops at the first true test

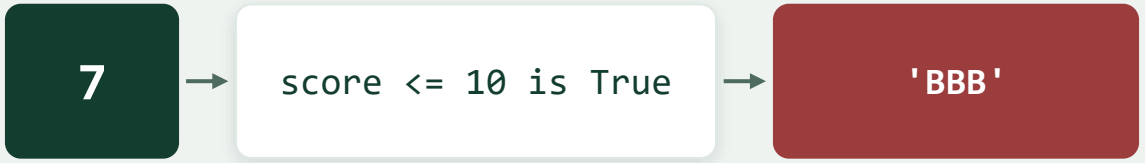
Function 1: narrowest test first

```
def bucket_one(score):
    if score <= 7:
        label = 'A or better'
    elif score <= 10:
        label = 'BBB'
    else:
        label = 'High yield'
    return label
```



Function 2: widest test first

```
def bucket_two(score):
    if score <= 10:
        label = 'BBB'
    elif score <= 7:
        label = 'A or better'
    else:
        label = 'High yield'
    return label
```



PLWK is rated A-, a score of 7. When conditions overlap, put the narrowest one first.

One object for the whole portfolio

Six separate lists

```
ticker_list
sector_list
score_list
coupon_list
price_list
face_list
```

One dictionary

```
portfolio = {
    'Ticker': ticker_list,
    'Sector': sector_list,
    'Score': score_list,
    'Coupon': coupon_list,
    'Price': price_list,
    'Face': face_list
}
```

Key first, then position

```
portfolio['Ticker']
```

The whole column of tickers

```
portfolio['Ticker'][2]
```

'PLWK', the third ticker

Each key is a column name. Each value is the list for that column.

From a dictionary to a table

```
import pandas as pd
df = pd.DataFrame(portfolio)
```

	Ticker	Sector	Score	Coupon	Price	Face
0	QVNE	Energy	9	5.250	98.50	1500000
1	DNMR	Transportation	11	6.125	102.75	500000
2	PLWK	Materials	7	4.750	95.00	750000
3	HLVF	Consumer	14	7.500	100.50	250000
4	OSTV	Telecom	11	5.750	98.50	1000000
5	VYRH	Health Care	12	6.750	102.00	400000

The keys became column names, and each list became a column. Two assert checks tie the total face, 4,400,000, to the email: once on the lists, once on the table.

`df.head(3)`

The first 3 rows

`df.tail(3)`

The last 3 rows

`df.shape`

Rows and columns: (6, 6)

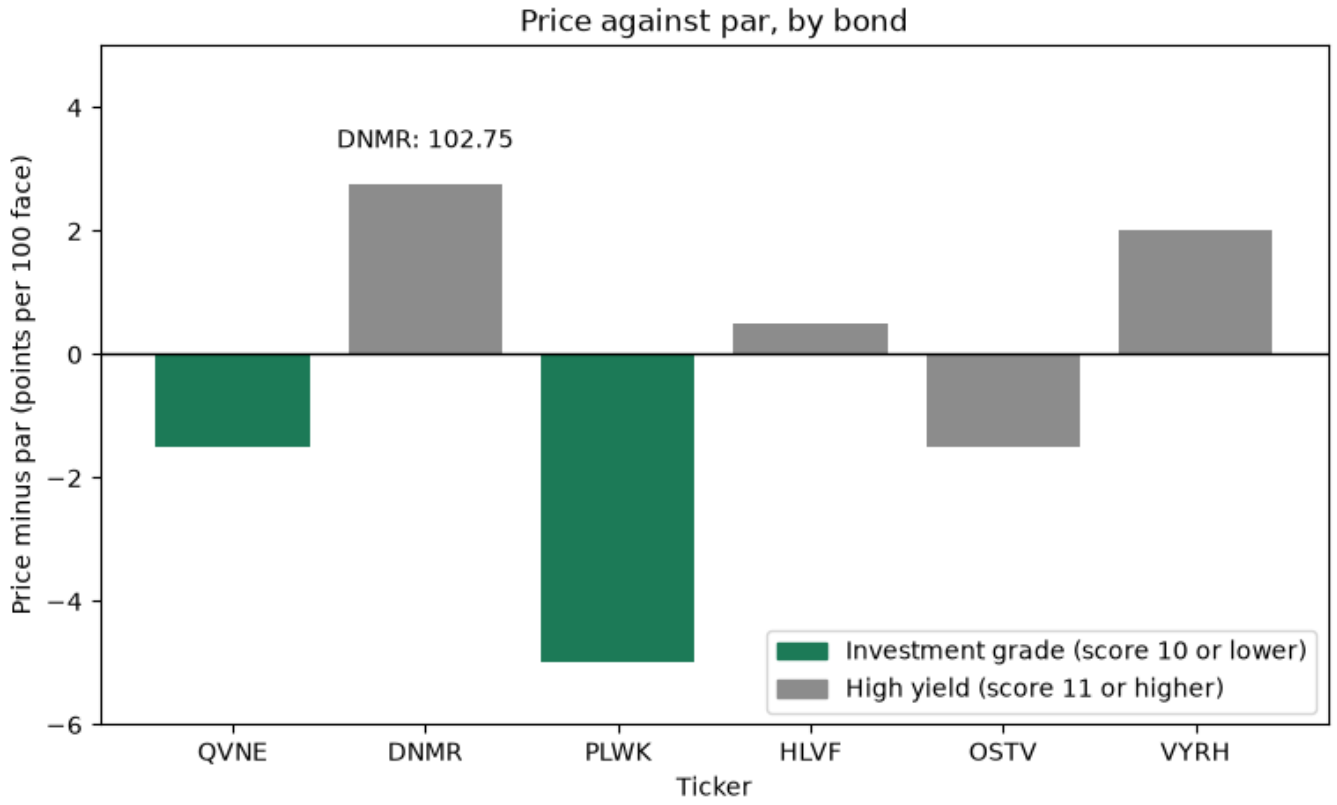
`df.info()`

Each column, its type, its count

`df.describe()`

A summary of every numeric column

The summary, as one picture



Above the line

Priced above par: DNMR, HLVF, VYRH

Below the line

Priced below par: QVNE, PLWK, OSTV

Grey bars

High yield: four of the six bonds

Week 3 teaches charts. For now, read the picture and check it against the table.

The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

Read

Read the code before you run it.

2

Predict

Decide what it should produce.

3

Run

Run it and look at the output.

4

Check

Compare with a number you worked out yourself.

Code that runs is not the same as code that is right.

What you can do now

- Look up and update any bond by its position in a list
- Apply a change to the right bonds with a condition inside a loop
- Count what you need, the way COUNTIF does
- Turn a lookup into a function, and return the answer
- Gather the lists into a dictionary and turn it into a DataFrame

Next week NumPy and pandas, and the portfolio grows from six bonds to two hundred